

Gestione degli errori

Gestione eccezioni avanzata

Tecniche avanzate per gestire le eccezioni in Python.

Ripasso rapido

Hai già visto come usare `try/except` per gestire gli errori. Ora vediamo tecniche più avanzate per situazioni più complesse.

```
try:
    # codice che potrebbe dare errore
except TipoErrore:
    # cosa fare se c'è quell'errore
else:
    # eseguito solo se NON ci sono stati errori
finally:
    # eseguito SEMPRE, con o senza errori
```

Creare i tuoi errori personalizzati

Python ti permette di creare **errori su misura** per il tuo programma. È utile quando vuoi segnalare situazioni specifiche con messaggi chiari.

Pensa a un modulo di iscrizione scolastica: ha senso creare un errore `EtaNonValidaError` invece di usare il generico `ValueError`.

```

# Creo un errore personalizzato per età non valide
class EtaNonValidaError(Exception):
    """Segnala che l'età inserita non è valida."""
    pass

# Creo un errore personalizzato per voti non validi, con informazioni extra
class VotoNonValidoError(Exception):
    """Segnala che il voto inserito non è nell'intervallo 0-10."""
    def __init__(self, voto, messaggio="Il voto deve essere tra 0 e 10"):
        self.voto = voto          # Salvo il voto errato per poterlo
        mostrare
        self.messaggio = messaggio
        super().__init__(self.messaggio) # Passa il messaggio alla classe
        madre

# Funzioni che usano gli errori personalizzati
def controlla_eta(eta):
    if eta < 0 or eta > 150:
        raise EtaNonValidaError(f"L'età {eta} non è valida.")

def controlla_voto(voto):
    if voto < 0 or voto > 10:
        raise VotoNonValidoError(voto)

# Utilizzo
try:
    controlla_eta(-5)
except EtaNonValidaError as e:
    print(f"Errore: {e}")
# → Errore: L'età -5 non è valida.

try:
    controlla_voto(15)
except VotoNonValidoError as e:
    print(f"Errore: {e.messaggio} (voto ricevuto: {e.voto})")
# → Errore: Il voto deve essere tra 0 e 10 (voto ricevuto: 15)

```

Lanciare un errore manualmente con **raise**

Puoi **provocare** tu stesso un errore quando ti accorgi che qualcosa non va, anche senza che Python lo faccia automaticamente.

```
def dividi(a, b):
    if b == 0:
        raise ValueError("Il divisore non può essere zero.") # Lancio
        l'errore
    return a / b

try:
    risultato = dividi(10, 0)
except ValueError as e:
    print(f"Errore: {e}")
# → Errore: Il divisore non può essere zero.
```

È come un cassiere che si rifiuta di accettare una banconota falsa: non aspetta che la banca se ne accorga — interviene subito.

Catturare un errore, fare qualcosa, e rilancerlo

A volte vuoi **registrare** che un errore è avvenuto (ad esempio scriverlo in un file di log), ma poi vuoi comunque che l'errore si propaghi al chiamante.

```
def leggi_file(percorso):
    try:
        with open(percorso) as f:
            return f.read()
    except FileNotFoundError as e:
        print(f"Log: il file '{percorso}' non è stato trovato.")
        raise # Rilancio lo stesso errore – non lo "ingoio"
```

Con `raise` da solo (senza argomenti), Python rilancia esattamente lo stesso errore che hai appena catturato.

Collegare gli errori tra loro (catena di eccezioni)

Puoi indicare che un errore è stato **causato** da un altro errore. Questo aiuta chi legge il messaggio di errore a capire l'intera catena di eventi.

```
def carica_configurazione(filename):
    try:
        with open(filename) as f:
            import json
            return json.load(f)
    except FileNotFoundError as e:
        # Lancio un errore più descrittivo, ma indico che è causato da "e"
        raise RuntimeError(f"Impossibile caricare la configurazione da '{filename}'") from e

try:
    config = carica_configurazione("config.json")
except RuntimeError as e:
    print(f"Errore: {e}")
    print(f"Causato da: {e.__cause__}")
```

La gerarchia degli errori

Gli errori in Python sono organizzati come un albero genealogico. Catturare un errore "genitore" significa catturare anche tutti i suoi "figli".

```
BaseException
├── Exception
│   ├── ArithmeticError
│   │   ├── ZeroDivisionError ← divisione per zero
│   │   └── OverflowError ← numero troppo grande
│   ├── LookupError
│   │   ├── IndexError ← indice lista fuori range
│   │   └── KeyError ← chiave dizionario non trovata
│   ├── ValueError ← valore sbagliato
│   ├── TypeError ← tipo sbagliato
│   └── OSError
│       └── FileNotFoundError ← file non trovato
```

```
try:
    x = [1, 2, 3][10]    # IndexError
except LookupError:
    # Cattura sia IndexError che KeyError, perché entrambi sono "figli" di
    # LookupError
    print("Errore di ricerca (indice o chiave non trovata)")
```

Context manager personalizzato (uso avanzato di `with`)

Hai già usato `with open(...)` per aprire file. Puoi creare i tuoi "context manager" che gestiscono automaticamente l'apertura e la chiusura di risorse — anche in caso di errore.

```
class GestoreRisorse:
    def __enter__(self):
        # Questo viene eseguito quando si entra nel blocco "with"
        print("Risorsa aperta")
        return self

    def __exit__(self, tipo_errore, valore_errore, traceback):
        # Questo viene eseguito quando si esce dal blocco "with" (anche in
        # caso di errore)
        print("Risorsa chiusa")
        if tipo_errore:
            print(f"Si è verificato un errore: {valore_errore}")
        return False # False = non "ingoia" l'errore, lascialo propagare

# Utilizzo
with GestoreRisorse() as r:
    print("Sto usando la risorsa")
    # Se qui avviene un errore, __exit__ viene comunque chiamato
```

Buone pratiche

```
# MALE: cattura tutti gli errori senza distinzione
# Nasconde bug e rende difficile il debug
try:
    operazione_complessa()
except:
    pass # Silenzio totale – non sai cosa è andato storto

# BENE: cattura solo gli errori che ti aspetti
try:
    operazione_complessa()
except (ValueError, TypeError) as e:
    print(f"Errore gestito: {e}")

# BENE: registra l'errore e poi rilancia
try:
    operazione_complessa()
except Exception as e:
    logging.error(f"Errore inatteso: {e}")
    raise # Non nascondere l'errore – rilanciarlo
```

La regola d'oro: **cattura solo gli errori che sai come gestire**. Per tutti gli altri, lascia che si propaghino e vengano visti.

Debugging di base

Come trovare e correggere gli errori nei programmi Python.

Perché ti serve fare debugging

Scrivere codice senza errori al primo colpo è praticamente impossibile — anche per i programmatori più esperti al mondo. Il **debugging** (o "debug") è l'arte di trovare e correggere gli errori nel tuo programma.

Pensa a un detective: quando qualcosa non funziona, devi raccogliere indizi, fare ipotesi e testare le tue teorie finché non trovi il colpevole.

I tre tipi di errori

1. Errori di sintassi

Sono come errori grammaticali: hai scritto qualcosa che Python non capisce. Vengono segnalati **prima ancora che il programma parta**.

```
# Manca i due punti alla fine della riga "if"
if x > 0
    print(x)
# SyntaxError: expected ':'
```

Sono i più semplici da correggere: Python ti dice esattamente dove si trova il problema.

2. Errori durante l'esecuzione

Il codice è scritto correttamente, ma qualcosa va storto **mentre il programma sta girando**.

```
lista = [1, 2, 3]
print(lista[10]) # Stai chiedendo il decimo elemento, ma ne esistono solo 3!
# IndexError: list index out of range
```

È come chiedere alla cassiera di un negozio il prodotto numero 10, ma sullo scaffale ce ne sono solo 3.

3. Errori logici

Questi sono i più insidiosi: il programma **gira senza errori**, ma produce risultati sbagliati. Python non se ne accorge — sei tu che devi trovarli.

```
def calcola_media(numeri):  
    return sum(numeri) / len(numeri) - 1 # Bug! Il "-1" non dovrebbe  
    esserci  
  
print(calcola_media([8, 7, 9])) # Stampa 7.0 invece di 8.0 – sbagliato!
```

Leggere il messaggio di errore (Traceback)

Quando Python trova un errore, stampa un **traceback** — una specie di "mappa" che ti mostra cosa stava succedendo nel momento dell'errore.

```
Traceback (most recent call last):  
  File "script.py", line 10, in <module>  
    risultato = dividi(10, 0)  
  File "script.py", line 4, in dividi  
    return a / b  
ZeroDivisionError: division by zero
```

Come leggerlo: parti dall'ultima riga e vai su.

- **Ultima riga:** il tipo di errore e la descrizione (`ZeroDivisionError: division by zero`)
- **Righe precedenti:** la sequenza di passaggi che ha portato all'errore (prima è stato chiamato `dividi`, che è a riga 10, e dentro `dividi` l'errore è a riga 4)

Tecnica 1: usare `print()` come spia

Il modo più semplice per capire cosa sta succedendo dentro il tuo programma è inserire delle `print()` temporanee per "spiare" i valori delle variabili.


```
def calcola_media(voti):
    print(f"DEBUG: voti ricevuti → {voti}")    # Spia: cosa arriva alla
    funzione?
    totale = sum(voti)
    print(f"DEBUG: totale calcolato → {totale}") # Spia: il totale è
    giusto?
    media = totale / len(voti)
    print(f"DEBUG: media calcolata → {media}")  # Spia: la media è
    giusta?
    return media

risultato = calcola_media([8, 7, 9])
print(f"Risultato finale: {risultato}")
```

Quando hai trovato il bug e corretto il codice, **ricordati di eliminare le righe di debug** prima di consegnare o condividere il programma.

Tecnica 2: il modulo **logging**

Le `print()` funzionano, ma hanno un problema: devi ricordarti di eliminarle tutte. Il modulo `logging` è più elegante: puoi **accenderlo o spegnerlo** con una sola riga.

```
# Questa riga "accende" tutti i messaggi di debug
logging.basicConfig(level=logging.DEBUG)

def calcola_media(voti):
    logging.debug(f"Voti ricevuti: {voti}")
    totale = sum(voti)
    logging.debug(f"Totale: {totale}")
    return totale / len(voti)

calcola_media([8, 7, 9])
```

I livelli di gravità, dal meno al più grave:

Livello	Quando usarlo
DEBUG	Dettagli per il debug
INFO	Informazioni generali ("il programma è partito")
WARNING	Qualcosa di strano, ma il programma continua
ERROR	Qualcosa è andato storto
CRITICAL	Errore gravissimo

Tecnica 3: il debugger interattivo `pdb`

Immagina di poter **mettere in pausa** il tuo programma nel mezzo dell'esecuzione e guardare dentro. È esattamente quello che fa `pdb`.

```
def funzione_problematica(x, y):
    breakpoint() # Il programma si ferma qui e aspetta i tuoi comandi
    risultato = x / y
    return risultato

funzione_problematica(10, 2)
```

Quando il programma si ferma, puoi scrivere comandi:

Comando	Significato
<code>n</code>	Esegui la riga successiva (next)
<code>s</code>	Entra dentro una funzione (step)
<code>p nome_variabile</code>	Stampa il valore di una variabile
<code>c</code>	Continua fino al prossimo punto di pausa (continue)
<code>q</code>	Esci dal debugger (quit)

Il comando `breakpoint()` è disponibile da Python 3.7 in poi. Nelle versioni precedenti si usava `import pdb; pdb.set_trace()`.

Tecnica 4: il debugger grafico (VS Code o PyCharm)

Se usi un editor come **VS Code** o **PyCharm**, hai a disposizione un debugger grafico molto più comodo di `pdb` :

- Clicca sul numero di riga per aggiungere un **punto di pausa** (breakpoint)
- Avanza riga per riga con i pulsanti
- Guarda tutte le variabili in una finestra apposita, senza dover digitare nulla

Trucchi utili

Controlla il tipo di una variabile

A volte il bug è che una variabile contiene un tipo di dato inaspettato (ad esempio una stringa invece di un numero).

```
x = qualche_funzione()
print(type(x), x)  # Stampa sia il tipo che il valore
```

Usa `assert` per aggiungere controlli

`assert` ti permette di scrivere "mi aspetto che questa condizione sia vera — se non lo è, fermati e dimmi perché".

```
def calcola_area(base, altezza):
    assert base > 0, f"La base deve essere positiva, ma ho ricevuto: {base}"
    assert altezza > 0, f"L'altezza deve essere positiva, ma ho ricevuto: {altezza}"
    return base * altezza

calcola_area(5, 3)      # Funziona, area = 15
calcola_area(-1, 3)    # AssertionError: La base deve essere positiva, ma ho ricevuto: -1
```

Isola il problema

Se il bug è in un programma lungo, crea un piccolo programma separato che riproduce solo il problema. Questo ti aiuta a:

- Eliminare tutto il codice non rilevante
- Concentrarti solo sul pezzo problematico
- Testare soluzioni più rapidamente

Gestione degli errori

Come evitare che il programma si blocchi quando qualcosa va storto.

Cosa succede quando si verifica un errore?

Ogni tanto il programma incontra una situazione che non riesce a gestire: l'utente scrive del testo quando ci si aspetta un numero, si cerca di dividere per zero, si prova ad aprire un file che non esiste.

In questi casi Python "lancia" un **errore** (chiamato tecnicamente *eccezione*) e, se non lo gestisci, il programma si blocca e mostra un messaggio di errore.

Esempi di errori comuni:

```
print(10 / 0)           # ZeroDivisionError – non si può dividere per
                        # zero
print("ciao" + 5)       # TypeError – non puoi sommare testo e numero
int("pippo")           # ValueError – "pippo" non è un numero
print(variabile_che_non_c_e) # NameError – variabile non definita

lista = [1, 2, 3]
print(lista[10])        # IndexError – la lista non ha la posizione 10
```

Il blocco `try / except`

Per evitare che il programma si blocchi, "avvolgi" il codice rischioso in un blocco `try`. Se si verifica un errore, Python salta al blocco `except` invece di fermarsi:

```
try:
    risultato = 10 / 0
except ZeroDivisionError:
    print("Non puoi dividere per zero!")
```

Traduzione in parole: "**Prova** a eseguire questo codice. Se si verifica un `ZeroDivisionError`, **invece** fai questo."

Gestire più tipi di errori

Puoi avere più blocchi `except`, uno per ogni tipo di errore che vuoi gestire:

```
try:
    numero = int(input("Inserisci un numero: "))
    risultato = 10 / numero
    print(f"10 / {numero} = {risultato}")
except ValueError:
    print("Devi inserire un numero valido, non del testo.")
except ZeroDivisionError:
    print("Non puoi dividere per zero.")
```

Il blocco `else` — "tutto è andato bene"

Il blocco `else` viene eseguito **solo se non si è verificato nessun errore**:

```
try:
    numero = int(input("Inserisci un numero: "))
    risultato = 10 / numero
except (ValueError, ZeroDivisionError) as e:
    print(f"Errore: {e}")
else:
    print(f"Risultato: {risultato}") # viene eseguito solo se tutto è ok
```

Il blocco **finally** — "sempre, in ogni caso"

Il blocco **finally** viene **sempre eseguito**, che ci sia stato un errore o no. Torna utile per fare "pulizia" (chiudere un file, liberare risorse):

```
try:
    file = open("esempio.txt", "r")
    contenuto = file.read()
except FileNotFoundError:
    print("File non trovato.")
finally:
    print("Operazione completata (con o senza errori).")
```

Leggere il messaggio di errore

Puoi salvare il messaggio di errore con **as** e stamparlo:

```
try:
    x = int("pippo")
except ValueError as e:
    print(f"Dettaglio errore: {e}")
    # Dettaglio errore: invalid literal for int() with base 10: 'pippo'
```

Gli errori più comuni

Errore	Quando succede
<code>ValueError</code>	Valore sbagliato per il tipo (es. <code>int("abc")</code>)
<code>TypeError</code>	Tipo sbagliato (es. <code>"ciao" + 5</code>)
<code>ZeroDivisionError</code>	Divisione per zero
<code>NameError</code>	Variabile non definita
<code>IndexError</code>	Indice fuori dalla lista
<code>KeyError</code>	Chiave non trovata nel dizionario
<code>FileNotFoundError</code>	File non trovato

Esempio pratico: input sicuro

```
while True:
    try:
        a = float(input("Numeratore: "))
        b = float(input("Denominatore: "))
        risultato = a / b
    except ValueError:
        print("Inserisci solo numeri validi.")
        continue
    except ZeroDivisionError:
        print("Non si può dividere per zero.")
        continue
    else:
        print(f"Risultato: {risultato}")
        break
```